

ToolShare

Code Review & Architecture Analysis

Category	Rating	Notes
Overall Architecture	Good	Clean MVC separation, well-structured layers
Route Organisation	Good	Logical grouping, auth middleware applied consistently
Controller Quality	Good	Clear, readable. Minor duplication across controllers
Model / Data Layer	Good	Thin models, clean SQL. Some raw queries in controllers
Error Handling	Adequate	HTTP codes are correct. Server errors lack detail
Security	Adequate	JWT auth is solid. Input validation could be centralised
Service Layer	Good	Scheduler, email, notifications cleanly separated
Code Consistency	Good	Uniform style throughout. Minor naming inconsistencies
Testability	Adequate	Structure supports testing. DB coupling is tight

1. Overview

ToolShare is a tool-lending platform built as a Node.js/Express API backed with a MariaDB (MySQL-compatible) relational database. The codebase follows a classic three-tier web architecture: routes handle HTTP concerns, controllers contain business logic, and models encapsulate database access. A small service layer sits alongside these tiers for concerns like email delivery, push notifications, and scheduled jobs.

The project is an application with roughly 20 source files across five logical layers. For a project of this scope, the code is well-organised and clearly written. A developer unfamiliar with the codebase could understand the request lifecycle for any given endpoint in a few minutes of reading.

2. Architecture

2.1 Layer Diagram

The application follows this request path:

HTTP Request
Express Router (routes/auth.js, routes/items.js, routes/bookings.js, routes/notifications.js, routes/admin.js)
Middleware (middleware/auth.js - JWT verification, role checks)
Controller (controllers/*.js - business logic, validation, orchestration)
Model (models/*.js - SQL queries)
MySQL / MariaDB Database

A services layer (services/) sits on this stack and is called from controllers. It handles email delivery (emailService.js), in-app notifications (notificationService.js), password reset token management (passwordResetService.js), and a background scheduler (overdueScheduler.js).

2.2 Strengths of the Architecture

- Separation of concerns is respected throughout. Routes declare endpoints and wire middleware. They contain no business logic. Controllers own logic and delegate persistence to models. Models contain only queries.
- The services layer correctly isolates infrastructure concerns. Email sending and notification creation are fire-and-forget calls that do not block the HTTP response, which is the right design for these operations.
- The overdue scheduler is cleanly extracted into its own module with two exported functions: autoMarkOverdue (which can be called on-demand from controllers) and startScheduler (called once at server boot). This dual interface is a good design choice.

- `server.js` is minimal and readable. It wires middleware, registers route groups, and starts the scheduler. Application logic does not leak into this file.

2.3 Architectural Concerns

- Some controllers bypass the model layer for one-off queries using a raw `query()` import (very importantly this includes `adminController.js` and `bookingController.js`). This breaks the clean model boundary and means that some SQL lives in two different places.
- There is no middleware layer for request validation (such as `express-validator`). All field validation is done inline at the top of each controller function. This is functional but leads to repetition and makes the validation rules harder to test, because it may not be immediately obvious where something breaks.

Limitations: Some controllers (admin, booking) bypass the model layer with raw SQL queries, breaking the clean model boundary. Input validation has no dedicated middleware layer, and it's all inline per controller, making it harder to test and maintain.

3. Route Organisation

Routes are split across five files, each mounted under `/api` in `server.js`. The grouping is very logical.

File	Prefix	Responsibility
<code>routes/auth.js</code>	<code>/api</code>	Registration, login, password reset, current-user fetch
<code>routes/items.js</code>	<code>/api</code>	Item CRUD, availability search, condition image upload
<code>routes/bookings.js</code>	<code>/api</code>	Borrow request lifecycle: create, approve, checkout, return
<code>routes/notifications.js</code>	<code>/api</code>	Fetch and mark notifications read
<code>routes/admin.js</code>	<code>/api/admin</code>	User management, category/faculty admin, reports

- Every route requires authentication via the `authenticateToken` middleware, which is consistently applied. The admin routes additionally put every endpoint behind a local `adminOnly` middleware function defined at the top of `routes/admin.js`. Any future admin route added to that file is automatically protected due to this.

Limitations: A concern is that all five route files are mounted at the same `/api` prefix in `server.js`, which means that route conflicts between files are possible. For example, if `routes/items.js` and `routes/bookings.js` both defined `GET /api/items`, only the first registered would respond. This is not currently a problem, but in the future if a new developer was brought on to expand the program, or we forgot about this while expanding in the future, this may create unintended issues.

4. Controllers

4.1 authController.js

Handles user registration, login, profile retrieval, and the two-step password reset process. The code is clean and appropriately concise. Notable design choices include:

- A makeToken helper at module scope keeps JWT signing logic in one place. If the token shape ever needs to change, there is exactly one location to update.
- The register endpoint correctly distinguishes between duplicate email and duplicate username errors by inspecting the MySQL error message string. This works, though it is sensitive to database engine changes. A better approach would be to check for the conflicting value before the insert.
- The forgot-password flow returns the same message regardless of whether the email exists, which is correct security practice as it prevents email enumeration attacks.
- Password reset token creation (via passwordResetService) correctly deletes any pre-existing token for the user before inserting a new one, preventing token accumulation in the database and also prevents old tokens from potentially being used.

4.2 itemController.js

Handles item and category management, including condition image uploads. This is the most complex controller. It is still readable, but it is doing more work than the others.

- Role gating (isStaff, isAdmin) is checked after field validation in most handlers, which means a student sending a malformed request to a staff-only endpoint gets a 400 rather than a 403. This is a minor inconsistency in behaviour.
- The serial number length cap (15 characters) is validated in two separate places. addItem and editItem with identical logic. This is a good candidate for a shared helper or middleware function.
- Condition image handling is split across four endpoints (by request ID and by item ID, each supporting GET and POST). The logic is correct but the duplication between uploadConditionImageForRequest and uploadConditionImageForItem is significant and could be refactored. This is done due to legacy concerns about implementation, and uncertainty over whether or not it would be a better idea to link condition images directly to their items or the borrow request they are connected to. Both are left in if a design change is made in the future.
- The autoMarkOverdue() call inside getAvailableItems is a side effect that happens during a read operation. This works, but it couples the availability query to the overdue scheduler in a non-obvious way. It is done to ensure availability results are accurate, which is a valid reason, but it still deserves to be mentioned.

4.3 bookingController.js

- This is the largest and most business-logic-heavy controller. It manages the full borrow request lifecycle across seven endpoints. Overall it is well-structured, but its size (over 250 lines) suggests it could benefit from some extraction.
- The bookItem and requestGroup endpoints both call autoMarkOverdue() before checking availability, which is a consistent way to do things.
- The requestGroup endpoint is notably complex, iterating over an array of items, performing individual conflict checks, handling per-item failures gracefully, and aggregating owner notifications. This logic is correct and the error aggregation pattern (returning both created_requests and failed_items) is a good API design choice.

- Status transition guards are handled inline in each endpoint rather than in a shared state machine. For example, **checkoutRequest checks that status === Approved, and returnRequest checks that status is CheckedOut or Overdue**. If a new status were added, each of these guards would need to be found and updated individually. A centralised transition map would be more maintainable.
- The controller imports a query directly from the database config for one lookup (the borrower name in requestGroup). This bypasses the User model and is kind of an ad-hoc query, although it is functional in the project's current state.

4.4 adminController.js

- Covers user management, category and faculty administration, request listing, and report generation. The controller is straightforward.
- The protection against deleting the default faculty (id === 1) is hardcoded as a magic number. Currently, the Film Faculty is hardcoded in as the default faculty. A more robust approach would be to mark the row in the database with a protected flag, so the constraint survives even if the faculty ID changes.
- The getReportsSummary endpoint executes three separate queries sequentially. These are independent and could be run in parallel with Promise.all(), which would reduce response time under load.
- Raw SQL commands for faculty operations live in the controller rather than in a Faculty model. A "Faculty model" does not exist, its logic is handled entirely by adminController.js. For consistency with the rest of the codebase, a Faculty model would be appropriate.

4.5 notificationController.js

- The simplest controller in the project, with three endpoints: fetch, mark-one-read, and mark-all-read. It is clean and correctly implemented. The only note is that markRead does not return a 404 if the notification ID does not exist or does not belong to the user, it silently succeeds. This is a minor issue but could be confusing to API consumers.

Limitations:

- **authController** - Minor issue. Duplicate email vs. username detection relies on parsing MySQL error strings, which is fragile.
- **itemController** - Serial number validation is duplicated between addItem and editItem. Role checks happen after field validation, causing inconsistent error responses (400 vs 403). Calling autoMarkOverdue() inside a read operation (getAvailableItems) is a non-obvious side effect.
- **bookingController** - Over 250 lines, could benefit from extraction. Status transition guards are duplicated inline across multiple endpoints rather than all being in one place. One raw database query bypasses the User model entirely.
- **adminController** - The default faculty is protected by a hardcoded magic number (id === 1) rather than a database flag. Three independent report queries run sequentially instead of in parallel.

Faculty SQL lives in the controller rather than a dedicated Faculty model.

notificationController - markRead silently succeeds even if the notification ID doesn't exist or belong to the user, rather than returning a 404.

5. Models

The models are thin wrappers around parameterised SQL queries, which is the right design for a project of this scale. They do not perform validation because that responsibility lives in the controllers, and they do not carry business logic. Each model exports a plain object of async functions.

5.1 BorrowRequest.js

- The most complex model, reflecting the central importance of borrow requests to the application. Key observations:
- hasConflict uses two different query shapes depending on whether an excludeId is provided, using a conditional expression to build both the SQL and parameters. This works, but is slightly hard to read. A cleaner approach would be to always include the exclusion clause, defaulting to an impossible value (e.g., -1) when no exclusion is needed.
- getOwnerHistory and getAdminFiltered both build dynamic SQL with conditional WHERE clauses. The approach is correct and uses parameterised values, avoiding SQL injection. The logic is readable.
- countActiveByBorrower is used in requestGroup to enforce a limit of 5 active bookings per borrower. This is a business rule that currently lives partly in the model (the COUNT query) and partly in the controller (the ≥ 5 check). Consolidating this into a model method would improve clarity.

5.2 User.js

- Clean and complete. Password hashing on create and updatePassword is done at the model level using bcrypt. The verifyPassword method is a thin wrapper around bcrypt.compare, which is the right abstraction. findById correctly excludes the password hash from its SELECT, which is a good security practice.

5.3 Item.js, Category.js, Notification.js, ConditionImage.js

- These models are all well-structured and appropriately scoped. A few notes:
- The Notification model includes an ALLOWED_TYPES set and sanitises the type parameter before inserting, defaulting to 'info' for unknown values. This is a defensive programming choice.
- Item.getAvailable uses a correlated NOT EXISTS subquery to exclude conflicting bookings. This is correct and efficient for small to medium datasets. At a larger scale, an indexed availability table would perform better.
- ConditionImage.getByItemId references image_id and timestamp as column aliases that come from the generated columns in the schema. This couples the model to the schema's generated column names, which could cause confusion if the schema changes.

Limitations: hasConflict in BorrowRequest.js uses a conditional query shape that's slightly hard to read. The active booking limit (5 per borrower) is split between the model (the COUNT query) and the controller (the ≥ 5 check), so the business rule isn't fully in one place. ConditionImage.getByItemId is coupled to schema-generated columns, which could cause confusion if the schema changes.

6. Services Layer

6.1 overdueScheduler.js

- Well-designed. The separation between `autoMarkOverdue` (the business logic) and `startScheduler` (the scheduling concern) is a good pattern as it allows controllers to call `autoMarkOverdue` on-demand without being coupled to the interval timer. The batch update (`markOverdueBatch`) is efficient, updating all overdue rows in a single query rather than one-by-one.
- One possible improvement: the scheduler does not handle the case where `autoMarkOverdue` throws an error mid-run. If the database is temporarily unavailable, the `setInterval` will continue firing and errors will be logged but the interval itself will not be affected. Adding a `try/catch` inside `startScheduler`'s interval callback would prevent unhandled promise rejections.

6.2 emailService.js

- A clean, straightforward nodemailer wrapper. The central `send` helper reduces boilerplate across the six specialised methods (`sendApproved`, `sendRejected`, etc.). The guard for missing to address (`if (!to) return Promise.resolve()`) is a good safety net.
- All email calls in controllers are fire-and-forget (`.catch(console.error)`), which is the right approach. A failed email should not fail the HTTP response. The email templates are inline strings, which is fine for a project of this scale.

6.3 notificationService.js

- A one-function module that wraps `Notification.create` in a fire-and-forget pattern. This is correct and clean. The abstraction means controllers never need to think about notification failure handling.

6.4 passwordResetService.js

- Manages password reset tokens in the database. The token is generated with 32 cryptographically random bytes, which provides entropy. Tokens expire after one hour, and old tokens are deleted before a new one is created, preventing duplicate token rows. Token verification uses a server-side expiry check (`expires_at > NOW()`), which is correct.

Limitations: The scheduler has no `try/catch` inside its interval callback, so if `autoMarkOverdue` throws (e.g. during a database outage), it will produce unhandled promise rejections. Email templates are inline strings, which is fine for now but would become harder to manage at scale.

7. Security

7.1 Authentication

- JWT-based authentication is implemented consistently. The `authenticateToken` middleware is applied to every protected route. Tokens are verified with `Jsonwebtoken` and the payload is attached to `req.user` for downstream use.
- The JWT secret falls back to an empty string if `JWT_SECRET` is not set, which means a misconfigured deployment would accept tokens signed with an empty secret. The environment variable should be required at startup rather than defaulted.

7.2 Authorization

- Role-based access control is implemented via the `isAdmin` and `isStaff` helpers in `middleware/auth.js`. Role checks are performed in each controller that requires them. This is functional, but it means the authorisation logic is distributed across controllers rather than centralised. A role-based middleware factory (eg, `requireRole('Faculty')`) applied at the route level would be cleaner and more auditable.

7.3 Input Validation

- Input validation is performed inline at the top of each controller function. Required field checks, date validation, and string length checks are all present. However, validation is not centralised, which means similar checks are duplicated across controllers. There is no schema-level validation library in use. This is a known trade-off for smaller projects but is worth addressing if the API surface grows.

7.4 SQL Injection

- All database queries use parameterised statements via the `mysql2` library's prepared statement interface. No string interpolation into SQL is present. The dynamic query builders in `BorrowRequest.getOwnerHistory` and `getAdminFiltered` correctly use placeholders for all user-supplied values. This is well-handled throughout

7.5 File Uploads

- File uploads are handled by `multer` with a 5MB size limit and a MIME type allowlist (JPEG, PNG, WebP). Files are stored on the local filesystem under the `uploads/` directory. For a production deployment, a cloud storage service (S3, GCS, etc.) would be more appropriate, but the current implementation is correct for a local or single-server deployment. The filename is generated using `Date.now()` + 12 random bytes, which avoids collisions.

Limitations: `JWT_SECRET` falls back to an empty string if not set, meaning a misconfigured deployment would accept tokens signed with no secret. This should cause a startup failure instead. Role-based authorization logic is spread across individual controllers rather than centralised at the route level. Input validation is duplicated and not schema-driven. File uploads go to the local filesystem rather than cloud storage, which isn't suitable for production.

8. Code Quality & Consistency

8.1 Style & Readability

- The codebase is consistently formatted and easy to read. Variable names are descriptive. Async/await is used uniformly, with try/catch error handling in every controller method. Comments are sparse but the code is generally self-documenting at the function level. A few of the more complex functions (requestGroup, getOwnerHistory) would benefit from inline comments explaining the intent of individual steps.

8.2 Naming Conventions

- Naming is mostly consistent. Models use PascalCase (BorrowRequest, ConditionImage), controllers and services use camelCase, and route files use lowercase. There are minor inconsistencies in field naming: the borrow request model uses both request_id and request_group_id (snake_case), while the JWT payload uses userId (camelCase). These inconsistencies are not bugs but can cause confusion when tracing data through the stack, and should likely be addressed before any future development to avoid compounding the issue.

8.3 Error Handling

- HTTP status codes are used correctly throughout: 400 for bad input, 401 for missing auth, 403 for insufficient permissions, 404 for missing resources, 409 for conflicts, and 500 for unhandled server errors. The global error handler in server.js catches multer-specific errors and returns appropriate messages.
- Server errors (status 500) return only the generic message 'Server error' to the client, which is correct from a security perspective. However, in development it would be helpful to also expose the error message behind a NODE_ENV check. All errors are logged to console.error, which is functional but a structured logging library (winston, pino) would be more appropriate for production.

8.4 Duplication

There are several instances of code duplication that are worth addressing:

- Serial number validation (max 15 characters) is duplicated in addItem and editItem.
- The autoMarkOverdue() call appears at the top of eight different controller functions. This is a side effect that could instead be applied as a route-level middleware on the relevant endpoints.
- Condition image upload logic is substantially duplicated between the by-request and by-item variants.
- Date range validation (parsing, NaN check, end > start check) is duplicated in bookItem and getAvailableItems. A shared validateDateRange helper would reduce this.

Limitations: Minor naming inconsistencies. snake_case in the database vs camelCase in JWT payload can cause confusion when tracing data through the stack. Server errors only return a generic "Server error" message with no detail even in development (should use a NODE_ENV check). No structured logging library in use, just console.error. Several instances of duplicated code: serial number validation, autoMarkOverdue() calls (appears in 8 places), condition image upload logic, and date range validation. All are candidates for shared helpers or middleware.

9. Recommendations

The following improvements are listed in approximate priority order:

High Priority

- Fail fast at startup if `JWT_SECRET` is not set. An empty secret is a critical security risk. Require `JWT_SECRET`.
- Replace the repeated `autoMarkOverdue()` calls in controllers with a single Express middleware applied to the relevant routes. This eliminates duplication and makes the dependency explicit. Centralise `autoMarkOverdue` as middleware.
- Move faculty-related SQL out of `adminController.js` and into a dedicated model for consistency with the rest of the codebase. Create a Faculty model
- In `getReportsSummary`, use `Promise.all([statusCounts, topTools, topBorrowers])` to run all three queries concurrently instead of sequentially. Run report queries in parallel.

Medium Priority

- Introducing an express-validator or a small validation middleware factory to reduce duplicated field checks across controllers. Centralise input validation:
- Define the valid transitions for borrow request status (Pending -> Approved, Approved -> CheckedOut, etc.) in a single place rather than inline in each controller. Add a status transition map.
- When `NODE_ENV` is not in production, include the error message in 500 responses to improve debuggability during development. Add error context in development.
- Merge the by-request and by-item upload handlers to reduce duplication, possibly accepting an optional `borrow_request_id` parameter. Refactor condition image uploads.

Lower Priority

- Mount route files at `/api/auth`, `/api/items`, `/api/bookings` etc. rather than all at `/api` to make the routing hierarchy explicit. Use explicit route sub-prefixes.
- Replace `console.error` with a library such as `pino` or `winston` for log levels, JSON output, and easier log aggregation. Add structured logging.
- Prevent unhandled rejections if `autoMarkOverdue` throws during a scheduled run. Add `try/catch` to the scheduler interval.
- For a production deployment, move uploaded files from the instance's local filesystem to a cloud storage service.

10. Conclusion

ToolShare's backend is a well-organised, readable Express API that correctly implements the MVC pattern. The layer boundaries are respected, the authentication and authorisation model is sound, and the borrow request lifecycle is handled with appropriate care for edge cases such as scheduling conflicts, status transition guards, and overdue marking.

The main areas for improvement are not correctness issues. The application seems to be functionally complete, but there are maintainability concerns. Duplicated validation and side-effect logic, a missing Faculty model, and an opportunity to centralise the status transition rules that are currently spread across the booking controller. Despite these findings, we believe the code is in a relatively good position. Many of these issues would not take long to fix and would simply require duplicating already existing code and making some relatively minor modifications, or taking code that is re-used in some places and abstracting it to a single function. The security and data integrity are fairly robust (which we believe are the most important), and this project should hold up under external attacks. Overall, we believe the quality of this code is very good.